

PAYMENT PROCESSING SYSTEM

NAME;ASHVITHA D

REG NO;23MIS0623

ENSURE PAYMENT ARE PROCESSED SECURELY AND CORRECTLY

1,DEPLOY PAYMENT VALIDATION LOGIC

PaymentServiceImpl.java

Java

```
package com.example.payment.service;
import com.example.payment.exception.PaymentException;
import com.example.payment.model.Transaction;
import com.example.payment.model.PaymentRequest;
import com.example.payment.repository.TransactionRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service @RequiredArgsConstructor
public class PaymentServiceImpl implements PaymentService {
    private final TransactionRepository repo;

    @Transactional
    public Transaction processPayment(PaymentRequest req) {
        // 1. Validate amount
        if (req.getAmount() == null || req.getAmount() <= 0)
            throw new PaymentException("Invalid payment amount");
        // 2. Validate card number (16 digits)
        if (!req.getCardNumber().matches("\\d{16}"))
            throw new PaymentException("Invalid card number");
        // 3. Check for duplicate request
        if (repo.existsByCardNumberAndAmountAndStatus(
            req.getCardNumber(), req.getAmount(), "SUCCESS"))
            throw new PaymentException("Duplicate payment request");
        // 4. Simulate balance check
        Transaction txn = new Transaction();
        txn.setCardNumber("*****" + req.getCardNumber().substring(12));
        txn.setAmount(req.getAmount());
        txn.setCurrency(req.getCurrency());
        if (req.getAmount() > req.getAccountBalance()) {
            txn.setStatus("FAILED");
            txn.setFailureReason("Insufficient funds");
        } else {
            txn.setStatus("SUCCESS");
        }
        return repo.save(txn);
    }
}
```

```

}
public Transaction getByld(Long id) {
return repo.findByld(id).orElseThrow(() ->
new PaymentException("Transaction not found: " + id));
}
@Transactional
public Transaction processRefund(Long id) {
Transaction txn = getByld(id);
if (!"SUCCESS".equals(txn.getStatus()))
throw new PaymentException("Only successful payments can be refunded");
txn.setStatus("REFUNDED");
return repo.save(txn);
}
public List<Transaction> getAll() { return repo.findAll(); }

```

```

1 Payment Processing System - Project Report
2 Page 5PPS Report | CSE Dept
3 5. ENTITY - Transaction.java
4 JPA Entity / Database Model
5 Java
6 package com.example.payment.model;
7 import jakarta.persistence.*;
8 import lombok.Data;
9 import java.time.LocalDateTime;
10 @Entity @Table(name = "transactions") @Data
11 public class Transaction {
12     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14     private String cardNumber; // masked: last 4 digits only
15     private Double amount;
16     private String currency; // USD, INR, EUR
17     private String status; // SUCCESS, FAILED, REFUNDED
18     private String failureReason; // null if success
19     private LocalDateTime createdAt;
20     @PrePersist
21     pub

```

2,JUNITTEST FOR TRANSACTION

Payment Processing System — Project Report

Page 8PPS Report | CSE Dept

9. JUNIT TEST CASES

PaymentServiceTest.java — Unit Tests

Java

```

package com.example.payment;
import com.example.payment.exception.PaymentException;
import com.example.payment.model.*;
import com.example.payment.repository.TransactionRepository;
import com.example.payment.service.PaymentServiceImpl;

```

```

import org.junit.jupiter.api.*;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Optional;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {
    @Mock TransactionRepository repo;
    @InjectMocks PaymentServiceImpl service;
    private PaymentRequest validReq;
    @BeforeEach void setUp() {
        validReq = new PaymentRequest();
        validReq.setCardNumber("1234567890123456");
        validReq.setAmount(500.0); validReq.setCurrency("USD");
        validReq.setAccountBalance(1000.0);
    }
    @Test void testPayment_Success() {
        Transaction t = new Transaction(); t.setStatus("SUCCESS");
        when(repo.save(any())).thenReturn(t);
        assertEquals("SUCCESS", service.processPayment(validReq).getStatus());
    }
    @Test void testPayment_InsufficientFunds() {
        validReq.setAccountBalance(100.0);
        Transaction t = new Transaction(); t.setStatus("FAILED");
        when(repo.save(any())).thenReturn(t);
        assertEquals("FAILED", service.processPayment(validReq).getStatus());
    }
    @Test void testPayment_InvalidCard() {
        validReq.setCardNumber("123");
        assertThrows(PaymentException.class, () -> service.processPayment(validReq));
    }
    @Test void testPayment_NegativeAmount() {
        validReq.setAmount(-100.0);
        assertThrows(PaymentException.class, () -> service.processPayment(validReq));
    }
    @Test void testGetTransaction_Success() {
        Transaction t = new Transaction(); t.setId(1L);
        when(repo.findById(1L)).thenReturn(Optional.of(t));
        assertNotNull(service.getById(1L));
    }
    @Test void testRefund_Success() {
        Transaction t = new Transaction(); t.setStatus("SUCCESS");
        when(repo.findById(1L)).thenReturn(Optional.of(t));
        when(repo.save(any())).thenReturn(t);
        assertEquals("REFUNDED", service.processRefund(1L).getStatus());
    }
}

```

```
}  
@Te
```

3,IMPLEMENT CI/CD

```
ame: Payment CI/CD  
push: { branches: [main] }  
pull_request: { branches: [main] }  
jobs:  
  build-test-deploy:  
    runs-on: ubuntu-latest  
    services:  
      mysql:  
        image: mysql:8.0  
        env: { MYSQL_ROOT_PASSWORD: root, MYSQL_DATABASE: paymentdb }  
        ports: ['3306:3306']  
        options: --health-cmd="mysqladmin ping" --health-interval=10s  
    steps:  
      - uses: actions/checkout@v4  
      - name: Set up JDK 17  
        uses: actions/setup-java@v4  
        with: { java-version: '17', distribution: 'temurin' }  
      - name: Build & Run JUnit Tests  
        run: mvn clean verify --no-transfer-progress  
      - name: Build Docker Image  
        run: docker build -t ${{ secrets.DOCKER_USERNAME }}/payment-service:${{ github.sha }} .  
      - name: Push to Docker Hub  
        run: |  
          echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" --  
          passwor  
          docker push ${{ secrets.DOCKER_USERNAME }}/payment-service:${{ github.sha }}  
      - name: Deploy to Kubernetes  
        run: kubectl set image deployment/payment-service payment-service=${{  
        secrets.DOCKER_USERNAME }}/
```



4,DEPLOY ON DOCKER

DOCKER SETUP

Dockerfile & docker-compose.yml

Java

Dockerfile — Multi-stage build

FROM maven:3.9-eclipse-temurin-17 AS builder

WORKDIR /app

COPY pom.xml .

RUN mvn dependency:go-offline -q

COPY src ./src

RUN mvn clean package -DskipTests -q

FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

COPY --from=builder /app/target/payment-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]

HEALTHCHECK --interval=30s CMD wget -q --spider http://localhost:8080/actuator/health || exit 1

YAML

docker-compose.yml

version: '3.8'

services:

mysql:

image: mysql:8.0

environment: { MYSQL_ROOT_PASSWORD: root, MYSQL_DATABASE: paymentdb }

ports: ["3306:3306"]

volumes: [mysql-data:/var/lib/mysql]

healthcheck:

test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]

interval: 10s

payment-service:

build: .

ports: ["8080:8080"]

environment:

SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/paymentdb

SPRING_DATASOURCE_USERNAME: root

SPRING_DATASOURCE_PASSWORD: root

depends_on:

DEPLOY ON KUBERNETES

AML

k8s/deployment.yml

apiVersion: apps/v1

kind: Deployment

metadata:

name: payment-service

spec:

replicas: 2

```
selector: { matchLabels: { app: payment-service } }
template:
  metadata: { labels: { app: payment-service } }
  spec:
    containers:
      - name: payment-service
        image: yourdockerhub/payment-service:latest
        ports: [{ containerPort: 8080 }]
        env:
          - { name: SPRING_DATASOURCE_URL, value: jdbc:mysql://mysql-service:3306/paymentdb }
        readinessProbe:
          httpGet: { path: /actuator/health, port: 8080 }
          initialDelaySeconds: 30
        resources:
          requests: { memory: "256Mi", cpu: "250m" }
          limits: { memory: "512Mi", cpu: "500m" }
YAML
# k8s/service.yml
apiVersion: v1
kind: Service
metadata:
  name: payment-service
spec:
  selector: { app: payment-service }
  ports: [{ port: 80, targetPort: 8080 }]
  type: LoadBalancer
# Commands:
# kubectl apply -f k8s/
# kubectl get pods
# kubectl get svc payment-serv
```